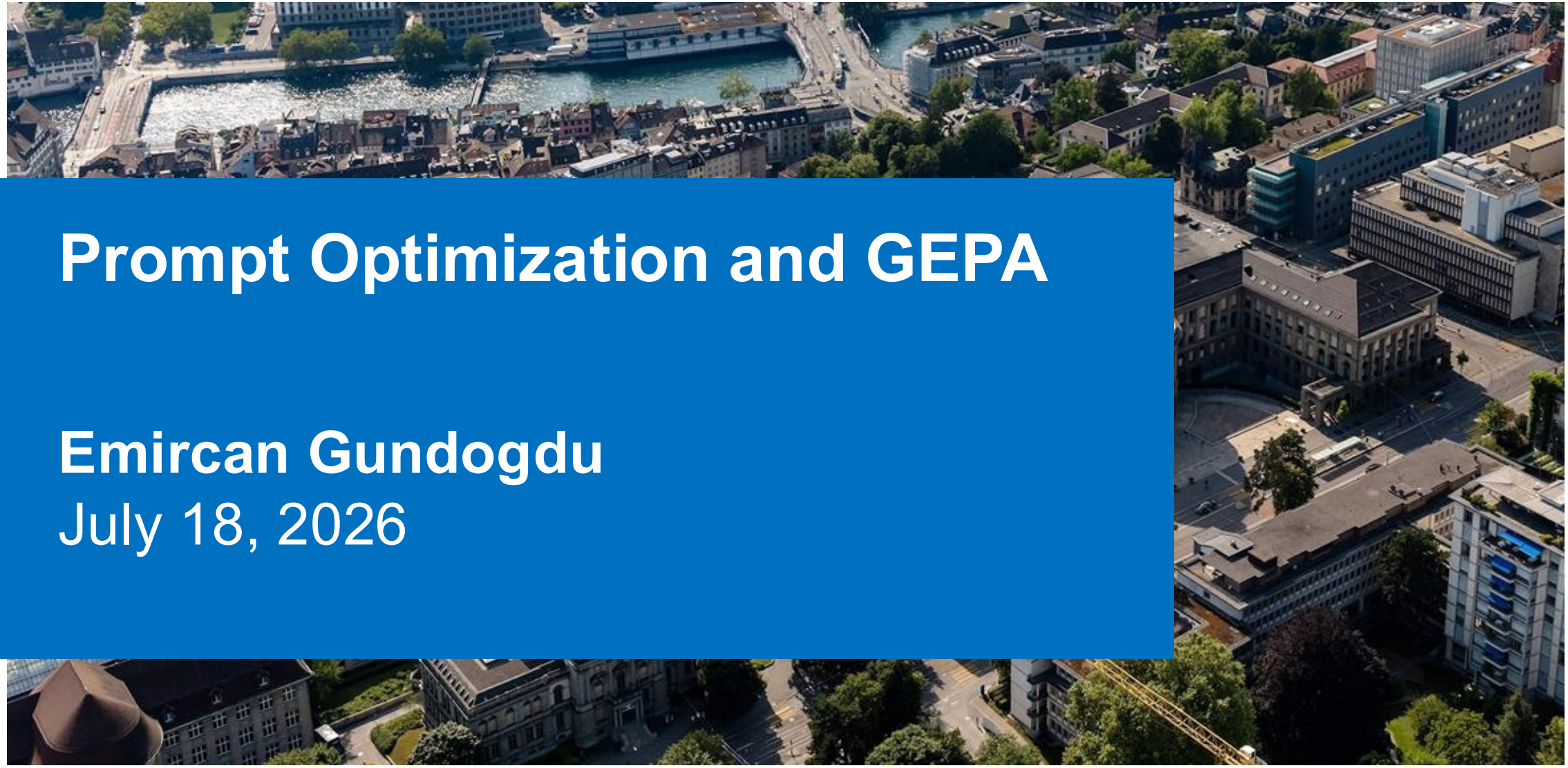
ETH zürich



Prompt Optimization and GEPA

Emircan Gundogdu

July 18, 2026

Agenda

Part 1. Why prompts matter

1. **The receiver problem:** same model, very different answers

Part 2. From engineering to optimization

2. **Prompt engineering:** few-shot, chain of thought
3. **Prompt optimization:** reinforcement learning, OPRO

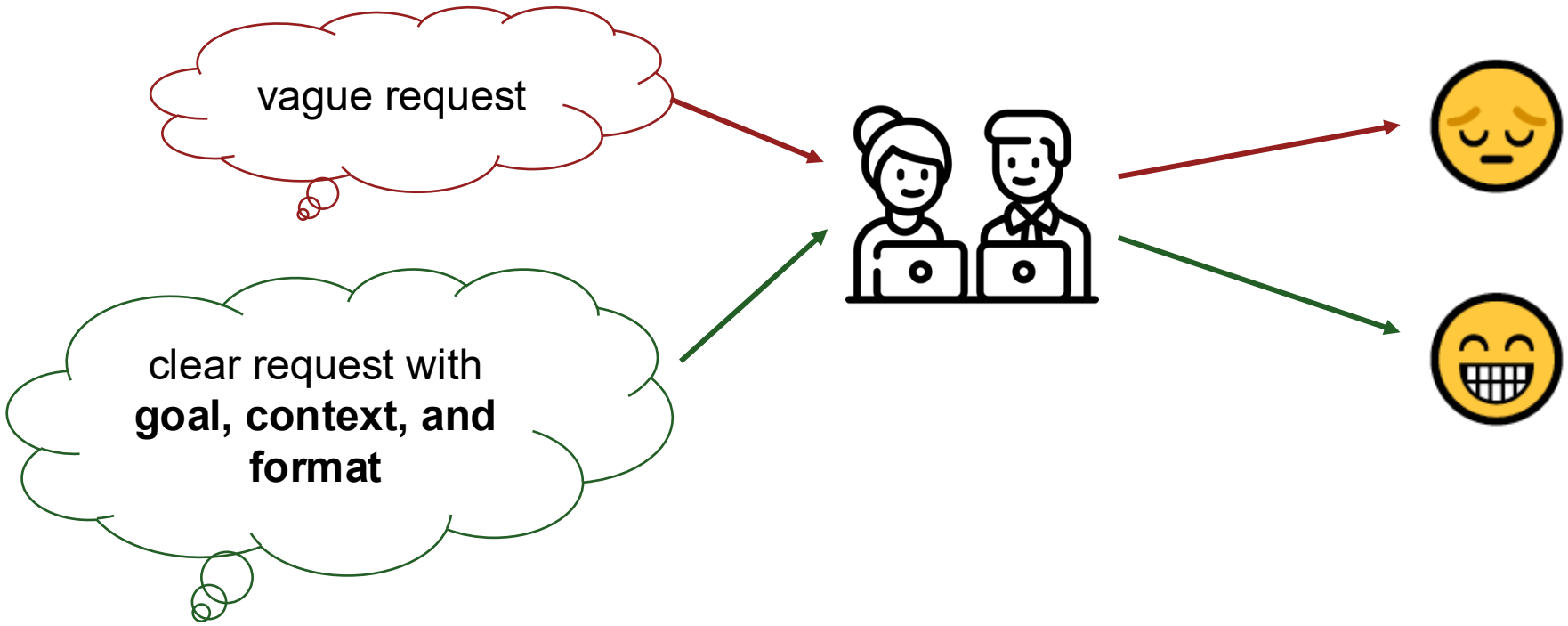
Part 3. GEPA

4. **Richer signals:** traces and feedback, not just a score
5. **The GEPA loop:** reflect, mutate, evaluate, Pareto-filter, merge
6. **Summary and takeaway**

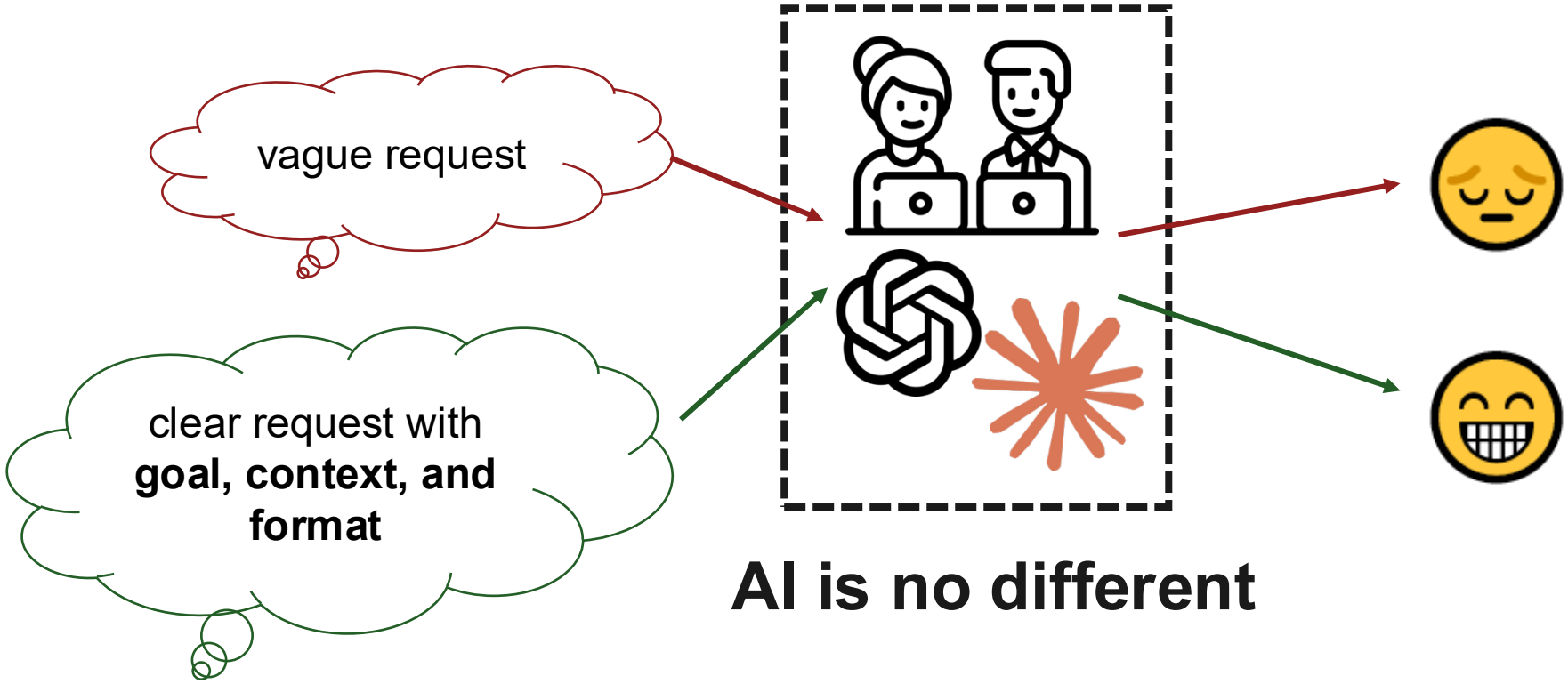
Outline

1. **Why prompts matter**
2. Prompt engineering
3. Prompt optimization
4. GEPA’s richer signals: traces and feedback
5. The GEPA loop
6. Summary and takeaway

Well, communication is also the sender's responsibility.



Well, communication is also the sender's responsibility.



AI is no different

Same model, different instruction

VAGUE PROMPT

“Summarize these customer comments.”

Gives a general summary.

SPECIFIC PROMPT

*“Group these customer comments into three groups. **Identify the most urgent issue** and **recommend one action** per group.”*

Gives priorities and actions.

Same model, different instruction

VAGUE PROMPT

“Summarize these customer comments.”

Gives a general summary.

SPECIFIC PROMPT

*“Group these customer comments into three groups. **Identify the most urgent issue** and **recommend one action** per group.”*

Gives priorities and actions.

A good prompt removes ambiguity.

Outline

1. Why prompts matter
2. **Prompt engineering**
3. Prompt optimization
4. GEPA’s richer signals: traces and feedback
5. The GEPA loop
6. Summary and takeaway

Prompt engineering

PROMPT ENGINEERING

You *write and refine the prompt* ***by hand****.*

Methods

- **Few-shot**
- **Chain-of-thought**

Prompt engineering: few-shot

- Show a few solved examples, then ask your question.
- The model copies the pattern.

PROMPT

“Great phone, love it!” → **Positive**
“Stopped working in a week.” → **Negative**
“Best purchase this year!” → ?

MODEL OUTPUT

Positive ✓ *it followed the pattern from the examples*

Prompt engineering: chain of thought

- Ask the model to think step by step, so **it shows its work** and gets the answer.

VAGUE

> **PROMPT**
>
> ```
> I had 17 apples, ate 5, and bought 8
> more. How many?
> ```
>
> **MODEL OUTPUT**
>
> **19** ✗ *just a guess*

Prompt engineering: chain of thought

- Ask the model to think step by step, so **it shows its work** and gets the answer.

VAGUE

PROMPT

I had 17 apples, ate 5, and bought 8 more. How many?

MODEL OUTPUT

19 ✗ *just a guess*

Think step by step.

Explain your reasoning.

Break the problem into smaller steps.

WITH CHAIN-OF-THOUGHT

PROMPT

I had 17 apples, ate 5, and bought 8 more. How many? **Think step by step.**

MODEL OUTPUT

17 − 5 = 12, 12 + 8 = 20

Answer: 20 ✓

Hand-crafting one good prompt is fine. But
what if you have **thousands** of tasks to solve?

Let an algorithm do it

Outline

1. Why prompts matter
2. Prompt engineering
3. **Prompt optimization**
4. GEPA’s richer signals: traces and feedback
5. The GEPA loop
6. Summary and takeaway

Prompt engineering and prompt optimization

PROMPT ENGINEERING

__You__ write and refine the prompt __by hand__.

Methods

- **Few-shot**
- **Chain-of-thought**

PROMPT OPTIMIZATION

__An algorithm__ searches for a better prompt __for you.__

Methods

- **Reinforcement learning**
- **OPRO**
- **GEPA**

Prompt optimization: reinforcement learning (RL)

- The model tries the task many times. Each try gets a **reward.**
- The system searches for prompts that **achieve higher scores.**

Prompt optimization: OPRO

- Two models work together. The **optimizer** writes prompts and the **evaluator** scores them.

OPTIMIZER
writes new prompts

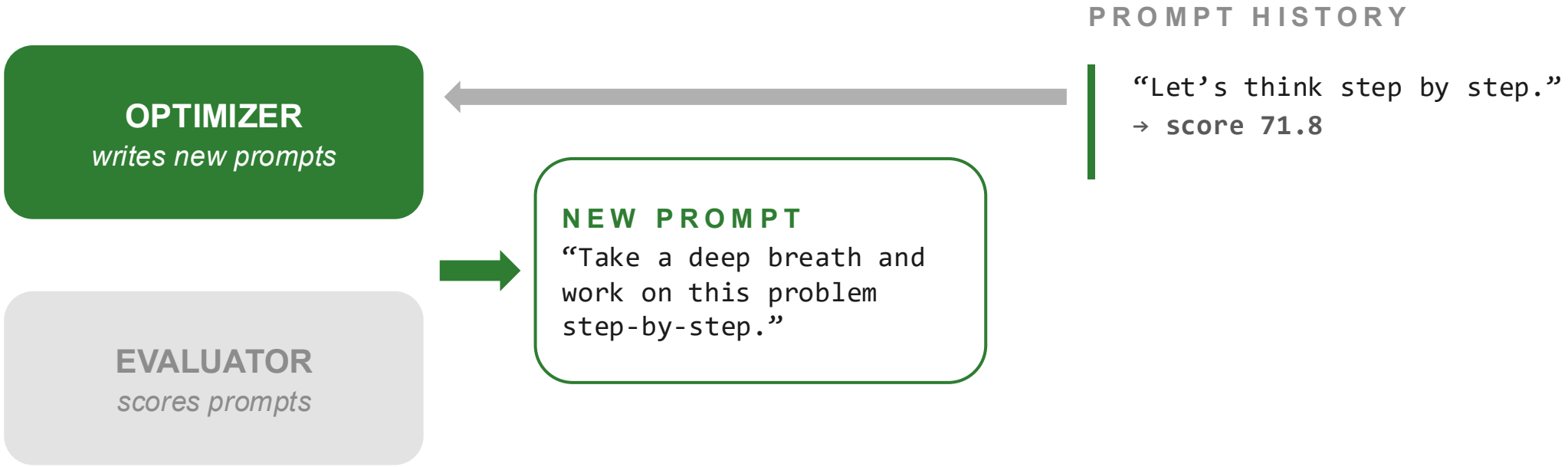
EVALUATOR
scores prompts

PROMPT HISTORY

> “Let’s think step by step.”
> → score 71.8

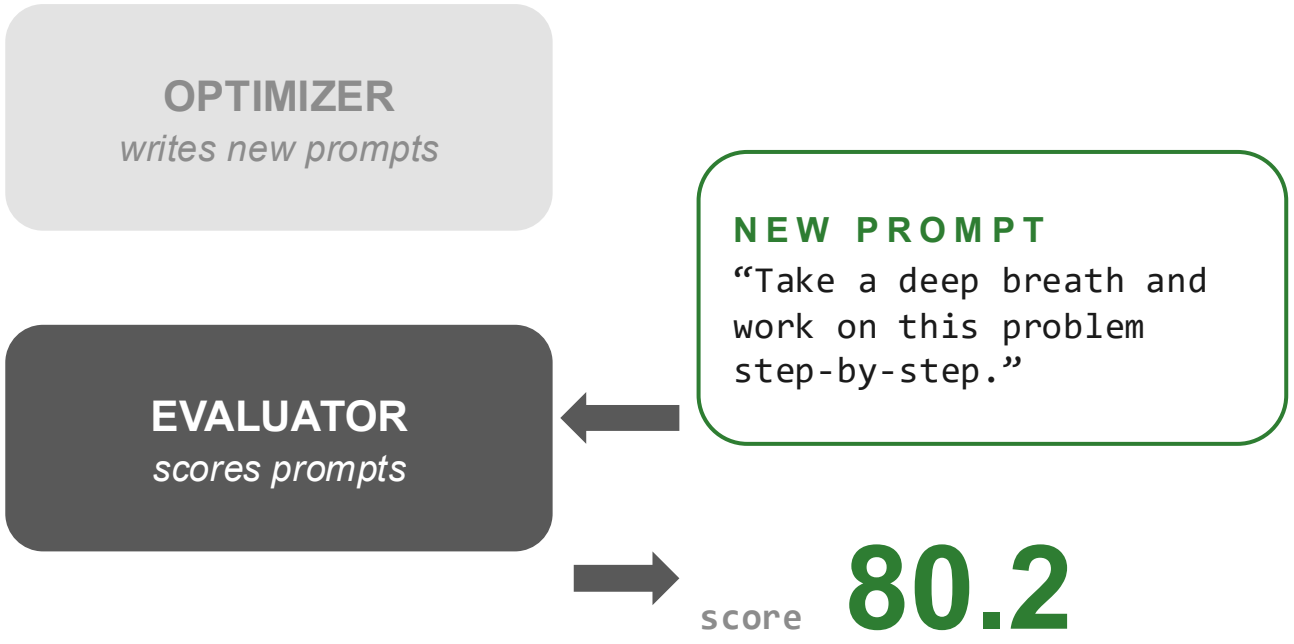
OPRO step 1: propose a new prompt

- The **optimizer** reads the history with scores and writes a new candidate prompt.



OPRO step 2: evaluate it

- The **evaluator** runs the new prompt on test questions and returns one number.



PROMPT HISTORY

> “Let’s think step by step.”
> → score 71.8

OPRO step 3: grow the history, repeat

- The scored prompt joins the history and the loop starts again.

Outline

1. Why prompts matter
2. Prompt engineering
3. Prompt optimization
4. **GEPA’s richer signals: traces and feedback**
5. The GEPA loop
6. Summary and takeaway

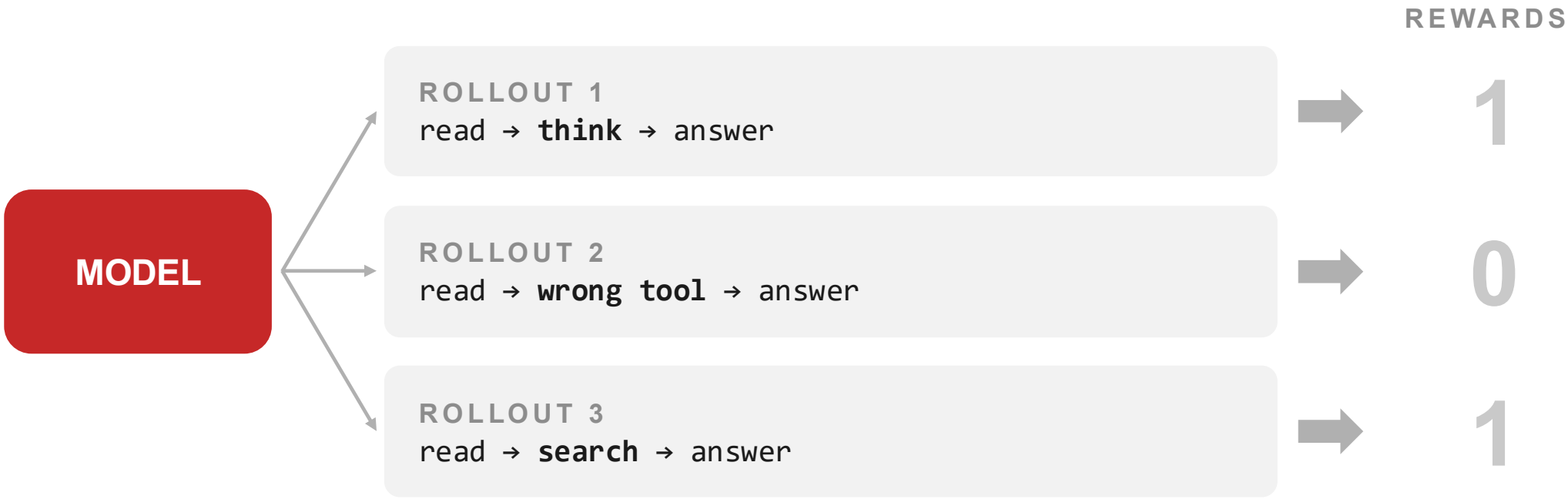
GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning

Lakshya A Agrawal[1], Shangyin Tan[1], Dilara Soylu[2], Noah Ziems[4],
Rishi Khare[1], Krista Opsahl-Ong[5], Arnav Singhvi[2,5], Herumb Shandilya[2],
Michael J Ryan[2], Meng Jiang[4], Christopher Potts[2], Koushik Sen[1],
Alexandros G. Dimakis[1,3], Ion Stoica[1], Dan Klein[1], Matei Zaharia[1,5], Omar Khattab[6]

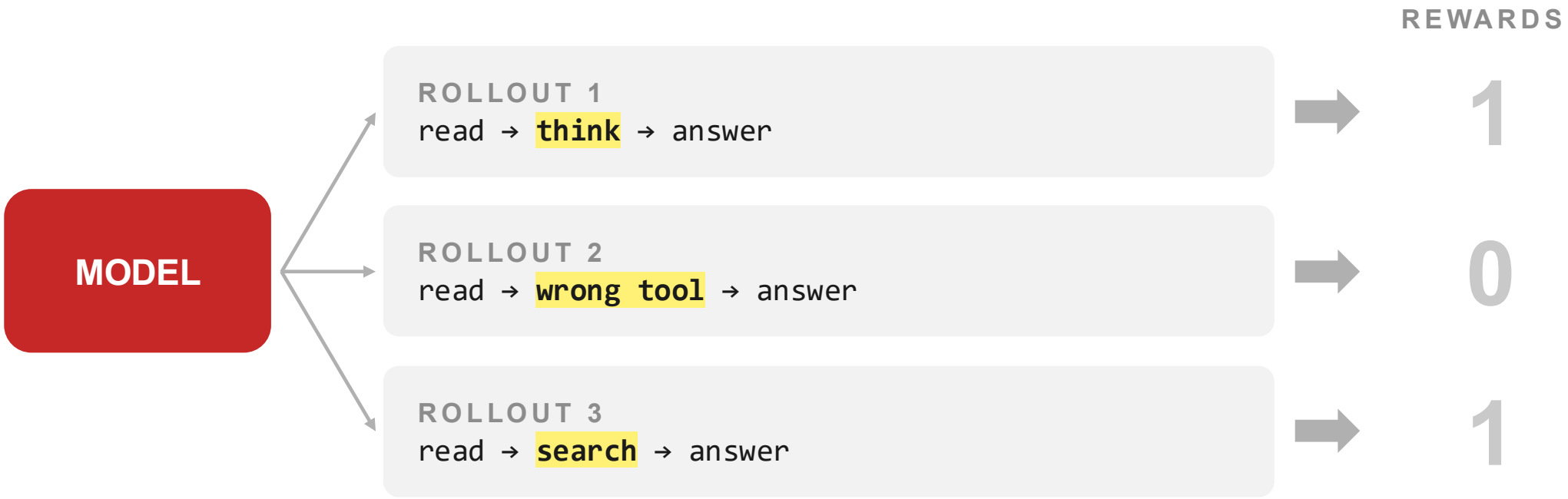
[1]UC Berkeley [2]Stanford [3]BespokeLabs.ai [4]Notre Dame [5]Databricks [6]MIT

- Published in Feb 2026.
- **It can outperform reinforcement learning**.

We have more than just the score/reward.



We have more than just the score/reward.

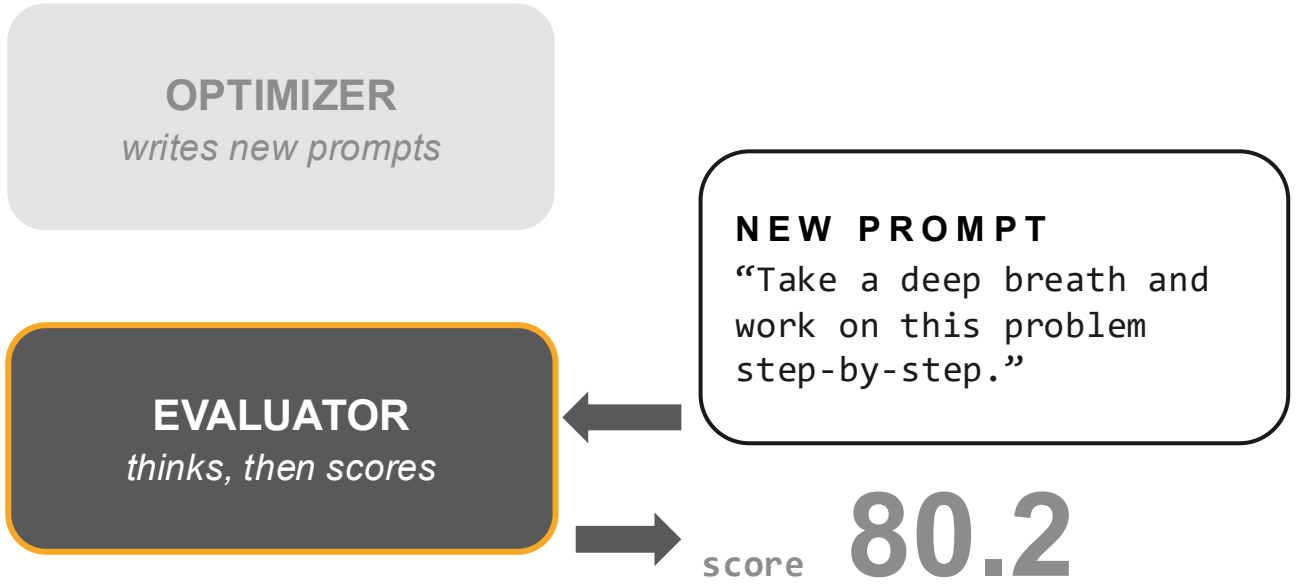


- Each rollout already contains how the model produce that reward.
- The score kept none of it.

We have intermediate results.

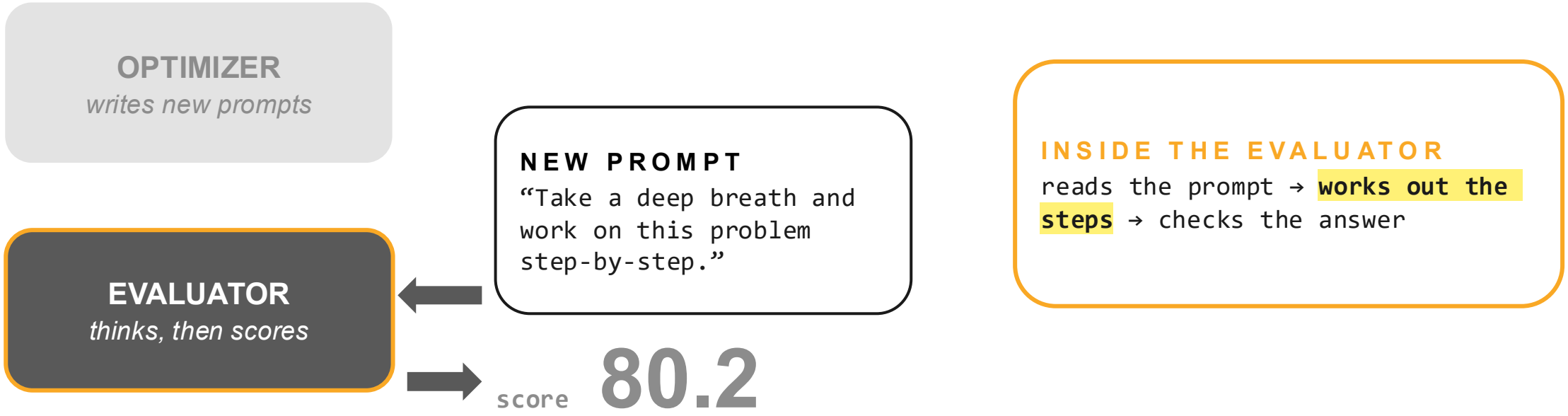
We have more than just the score/reward.

- Remember OPRO: to score a prompt, the **evaluator** works through the new prompt.



We have more than just the score/reward.

- Remember OPRO: to score a prompt, the **evaluator** works through the new prompt.



The score is only the final outcome.
The evaluator often generates valuable intermediate information.

This presentation

Honest takeaway

Optimization methods are just algorithms. **The information they use depends on the system design.**

Honest takeaway

Optimization methods are just algorithms. **The information they use depends on the system design.**

Same prompt: **“What is 12 + 10 − 2?”**

System A

`20`

answer only

Honest takeaway

Optimization methods are just algorithms. **The information they use depends on the system design.**

Same prompt: **“What is 12 + 10 − 2?”**

System A

20

answer only

System B

```
12 + 10 = 22
22 - 2 = 20
Answer: 20
```

shows its work

Honest takeaway

Optimization methods are just algorithms. **The information they use depends on the system design.**

Same prompt: **“What is 12 + 10 − 2?”**

System A

20

answer only

System B

```
12 + 10 = 22
22 − 2 = 20
Answer: 20
```

shows its work

Modern LLM systems can produce intermediate results. However, it is not guaranteed.

Their availability depends on:

- The model
- The prompt
- The system implementation

Trace

- A **trace** is the record of how an AI arrived at its answer: its reasoning, tool calls, intermediate results, and final output.

Trace

- A **trace** is the record of how an AI arrived at its answer: its reasoning, tool calls, intermediate results, and final output.

“When did we ship our first product?”

Trace

- A **trace** is the record of how an AI arrived at its answer: its reasoning, tool calls, intermediate results, and final output.

“When did we ship our first product?”

Reasoning “I need a year. Let me search the archive.” *the model thinks out loud*

Trace

- A **trace** is the record of how an AI arrived at its answer: its reasoning, tool calls, intermediate results, and final output.

“When did we ship our first product?”

Reasoning	“I need a year. Let me search the archive.”	*the model thinks out loud*
Tool call	`search(“first product ship year”)`	*it acts — calls a search tool*

Trace

- A **trace** is the record of how an AI arrived at its answer: its reasoning, tool calls, intermediate results, and final output.

“When did we ship our first product?”

Reasoning	“I need a year. Let me search the archive.”	*the model thinks out loud*
Tool call	**search(“first product ship year”)**	*it acts — calls a search tool*
Result	“Archive: first product shipped in 2011.”	*what the tool returned*
Answer	**“We shipped our first product in 2011.”**	*the final output the user sees*

But a trace never says if it was right

- The trace shows **what happened**, but nothing in it says whether the answer was good.

“When did we ship our first product?”

Reasoning	“I need a year. Let me search the archive.”
Tool call	search(“first product ship year”)
Result	“Archive: first product shipped in 2011.”
Answer	“We shipped our first product in 2011.”

Was “2011” even correct? **The trace cannot tell us, we need something that judges the answer.**

Feedback

- **Feedback** is what the grader says about the answer, and why.
- It can come from code, a grading rubric, or a person.

Feedback

- **Feedback** is what the grader says about the answer, and why.
- It can come from code, a grading rubric, or a person.

“We shipped our first product in 2011.”

Rubric “Incomplete: no product name, no source.” *graded against criteria*

Feedback

- **Feedback** is what the grader says about the answer, and why.
- It can come from code, a grading rubric, or a person.

“We shipped our first product in 2011.”

Rubric	“Incomplete: no product name, no source.”	*graded against criteria*
Code	AssertionError: expected ‘2011 – Quarter 1’	*an automatic check fails*

Feedback

- **Feedback** is what the grader says about the answer, and why.
- It can come from code, a grading rubric, or a person.

“We shipped our first product in 2011.”

Rubric	“Incomplete: no product name, no source.”	*graded against criteria*
Code	AssertionError: expected ‘2011 – Quarter 1’	*an automatic check fails*
Person	“Right year, but say which product.”	*a human reviews it*

Trace and feedback

- Every run may provide **up to two kinds of text,** neither is guaranteed.

Trace

Often available

Produced by the AI system.

Reasoning, tool calls, intermediate outputs.

Feedback

Optional

Produced by the evaluator.

Compiler errors, rubric comments, human explanations.

GEPA always uses **scores**.

It can additionally use **traces** and **feedback** when available.

Because language is a richer learning signal than a single number.

Outline

1. Why prompts matter
2. Prompt engineering
3. Prompt optimization
4. GEPA’s richer signals: traces and feedback
5. **The GEPA loop**
6. Summary and takeaway

GEPA

GENETIC

PARETO

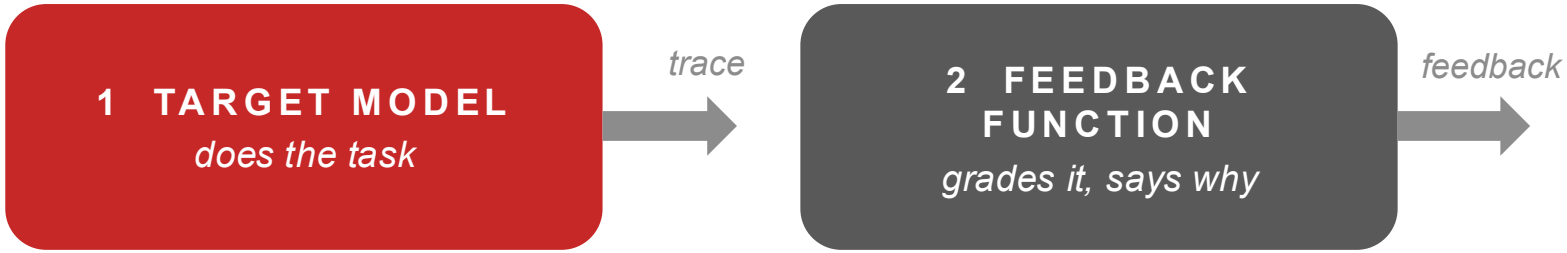
GEPA

- Three models pass plain text around a loop.

Runs the current prompt and produces the trace.

GEPA

- Three models pass plain text around a loop.



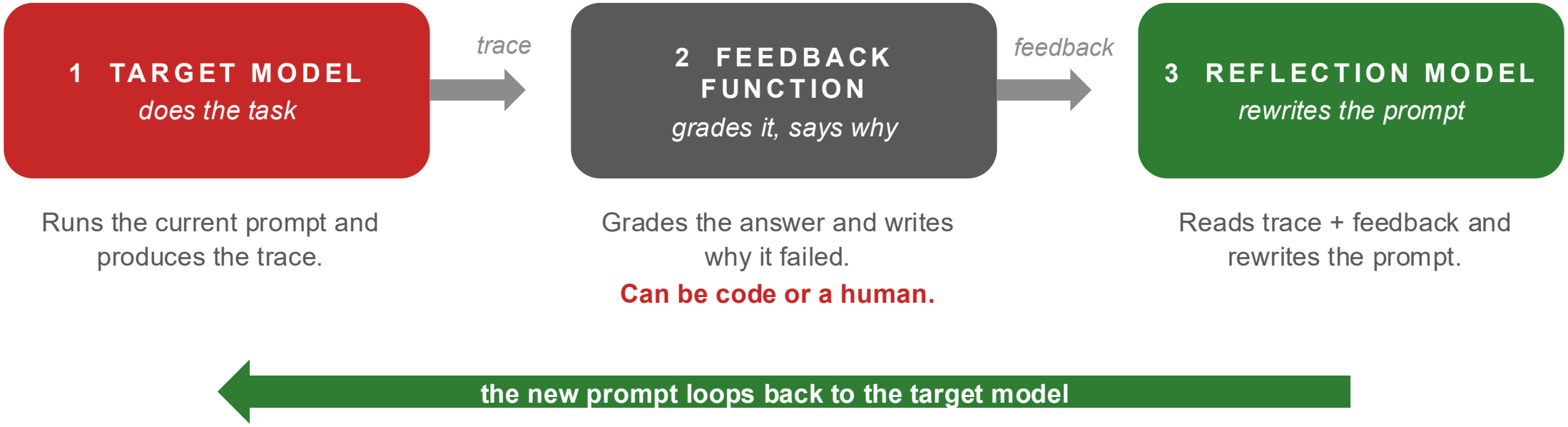
Runs the current prompt and produces the trace.

Grades the answer and writes why it failed.

Can be code or a human.

GEPA

- Three models pass plain text around a loop.



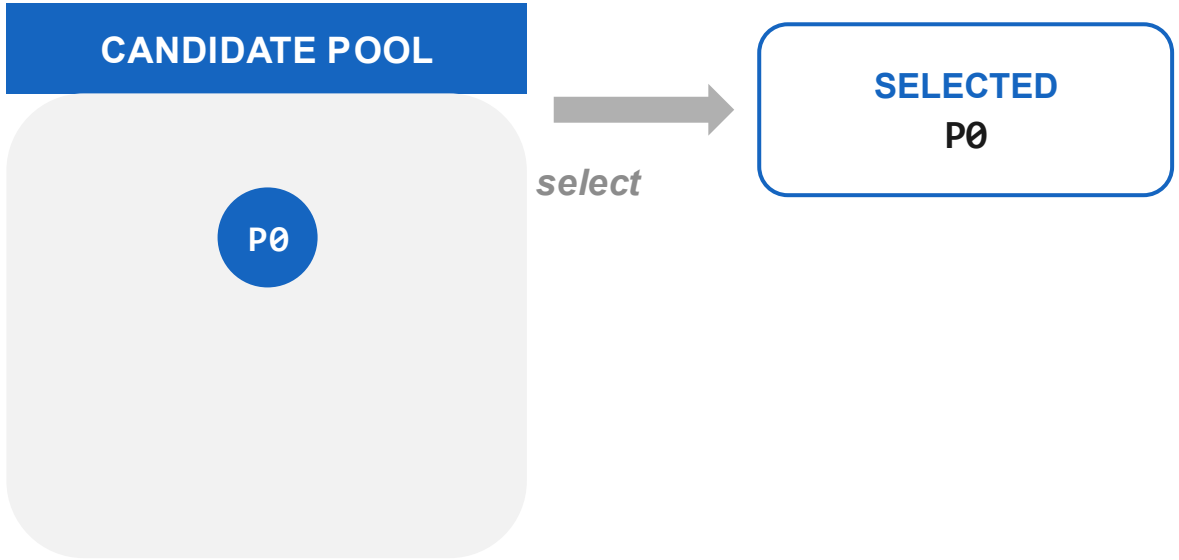
Step 1: the pool starts with one prompt

- GEPA keeps a **pool of candidate prompts**. At the start it holds exactly one: your base prompt.
- Everything grows from this single starting point.

P0: “Answer customer questions using the company knowledge base.”

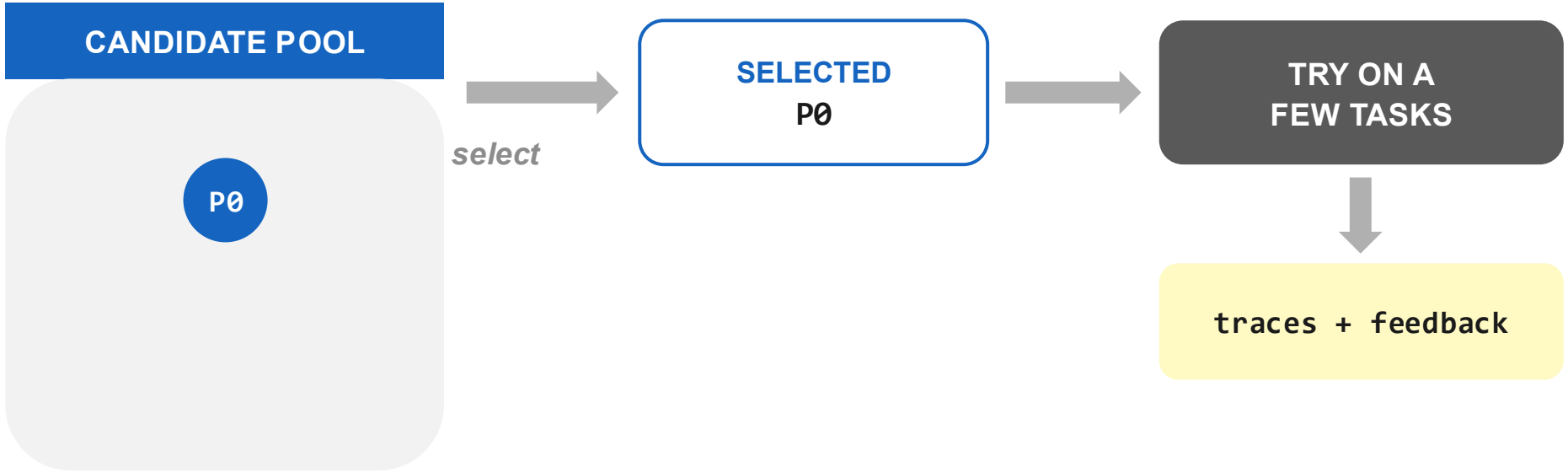
Step 2: select a prompt from the pool

- Each round, GEPA picks **one prompt** from the pool to improve.
- With one member, the choice is easy: we pick P0.



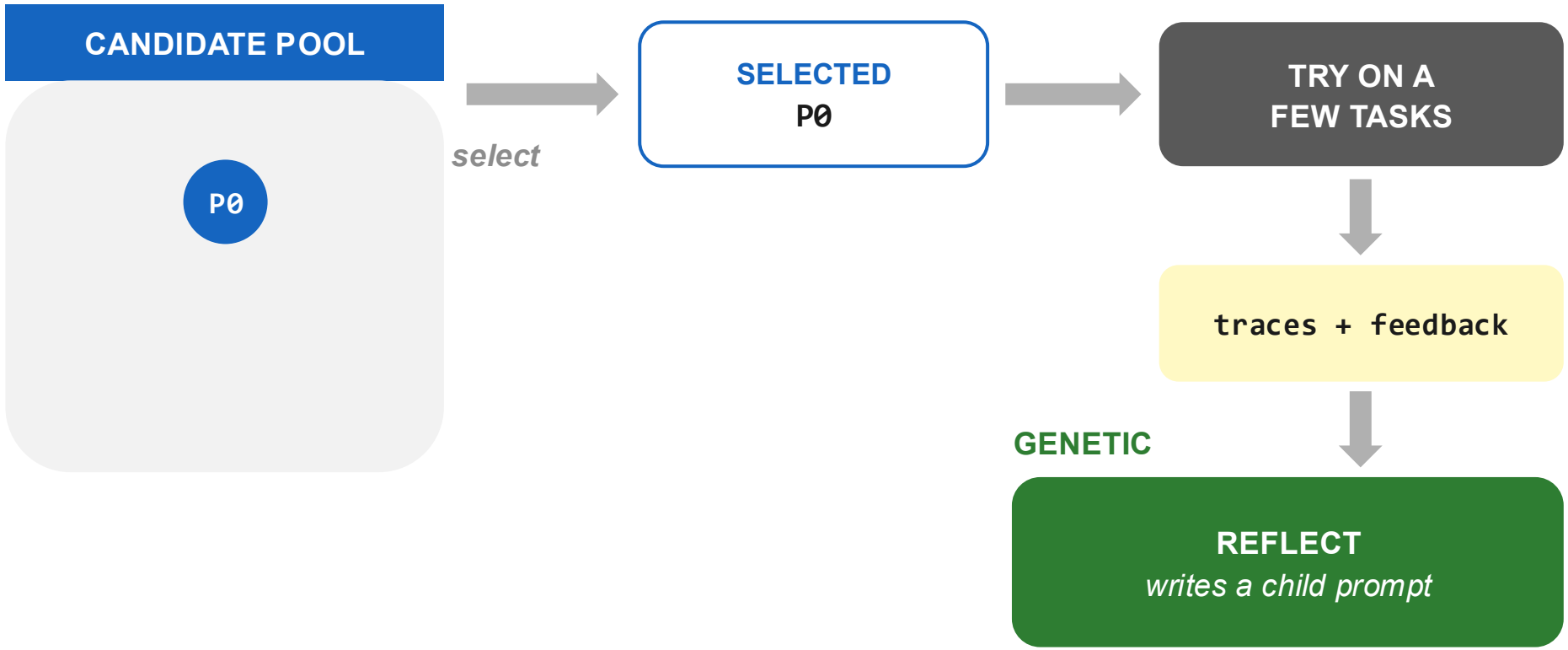
Step 3: try it, collect traces + feedback

- Run the selected prompt on a **handful of tasks**, not thousands.
- Capture the **traces** (what the AI did) and the **feedback** (why it failed).



Step 4: reflect, then mutate

- A **reflection model** reads the traces and feedback, and rewrites the prompt.



GEPA

GENETIC

Use text feedback to create better prompts.

1. MUTATION

improve one prompt

Take one prompt and rewrite it.

2. MERGE

combine two into one

Take two strong prompts and fuse them, keeping the best of each.

Mutation

- The **reflection model** reads the prompt, trace, feedback and score, and rewrites the prompt

SEED PROMPT

“Answer customer questions using the company knowledge base.”

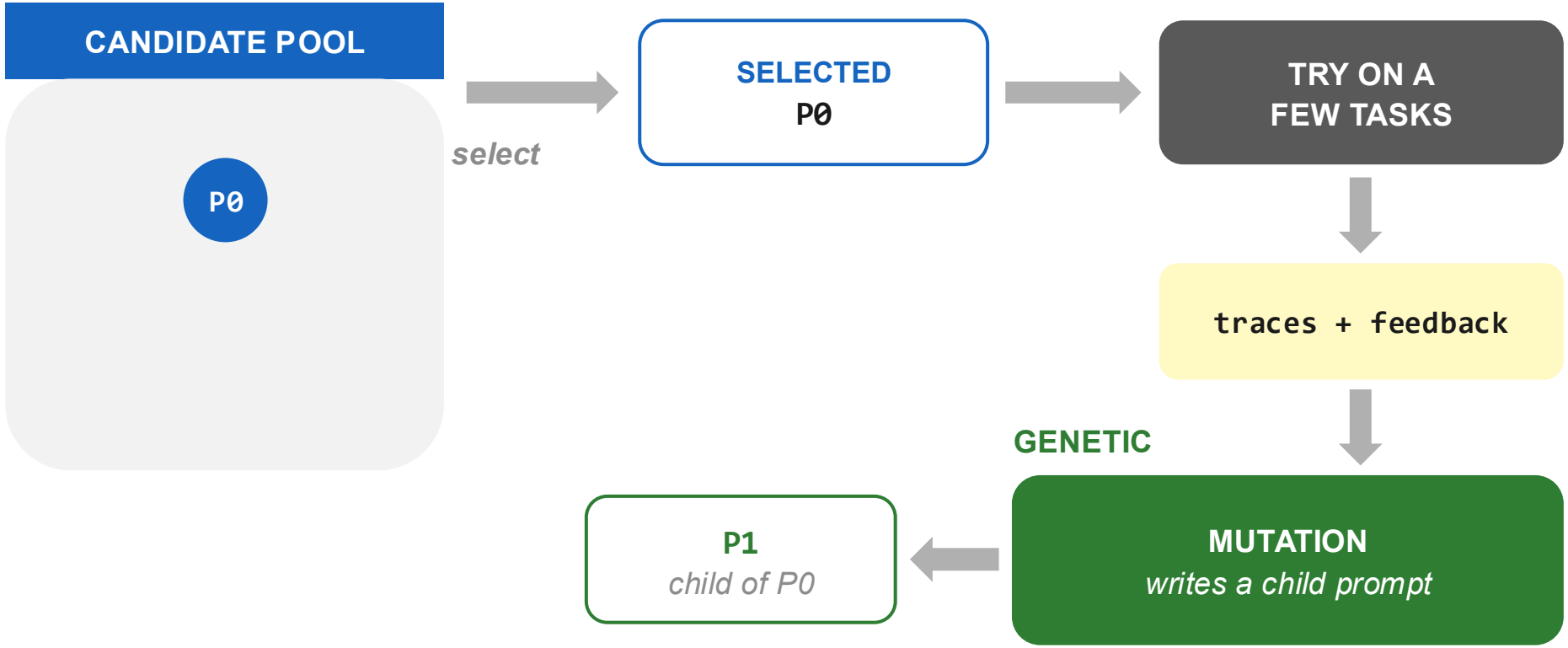
reflect on traces + feedback

MUTATED PROMPT

“Answer customer questions using the company knowledge base.
- Search the knowledge base before answering.
- If information is missing, ask a clarifying question.
- Cite the relevant document when possible.
- Explain your reasoning briefly.
- Do not invent information.”

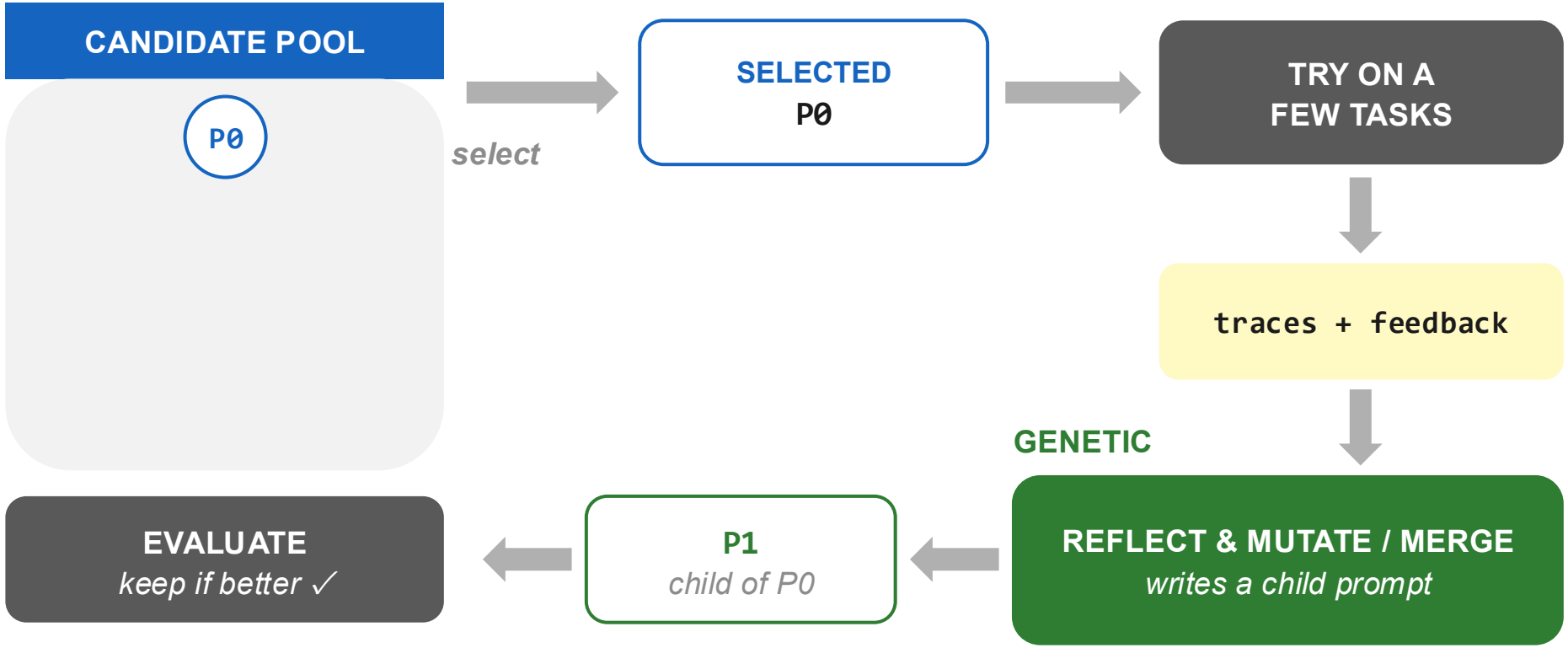
Step 4: reflect, then mutate

- A **reflection model** reads the traces and feedback, and rewrites the prompt into a child: P1.



Step 5: evaluate, keep if better

- We need to evaluate the new prompt.



Evaluate: score on a few tasks

- A **task** is one example we test the prompt on. We use a few so we see strengths, not one lucky score.

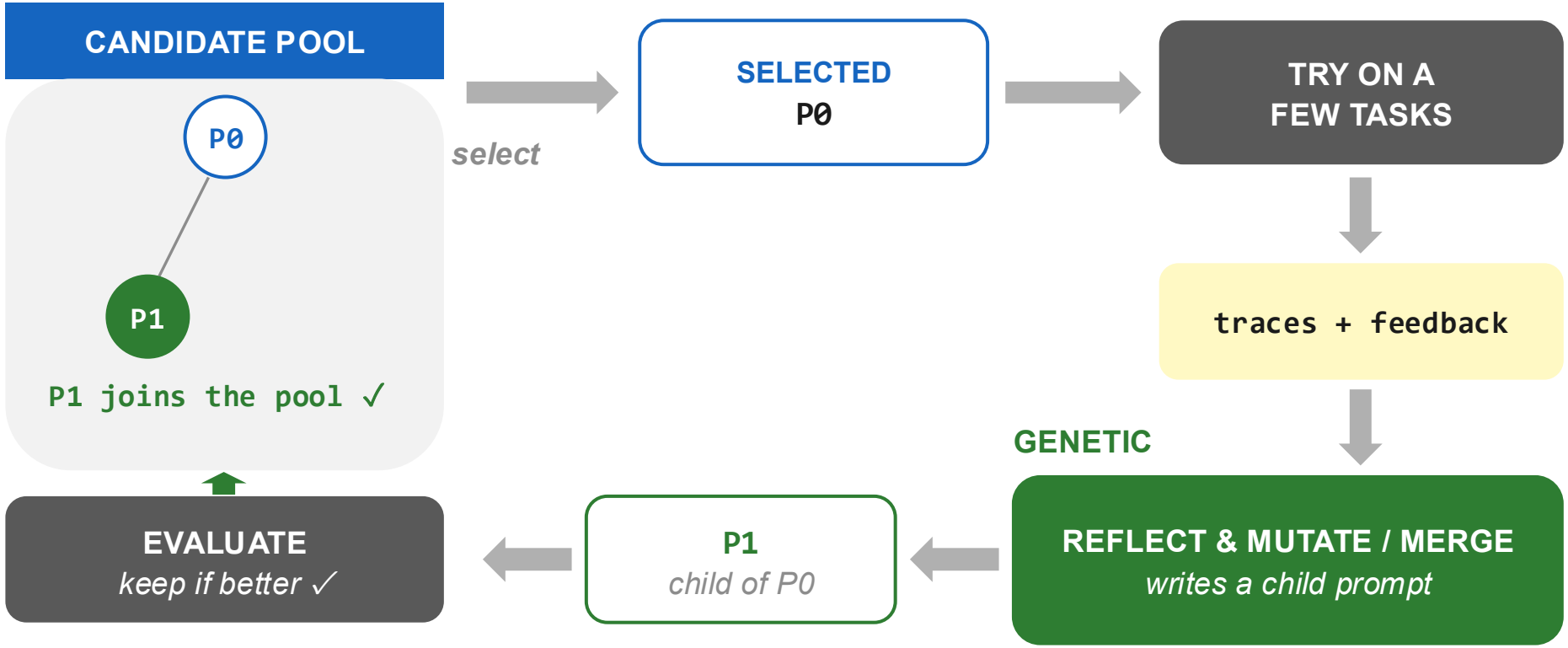
PROMPT	Task 1	Task 2	Task 3
P0	0.6	0.4	0.7
P1	0.5	0.8 ▲	0.7

P1 wins on at least one task.

That's enough to keep it even if it isn't best everywhere.

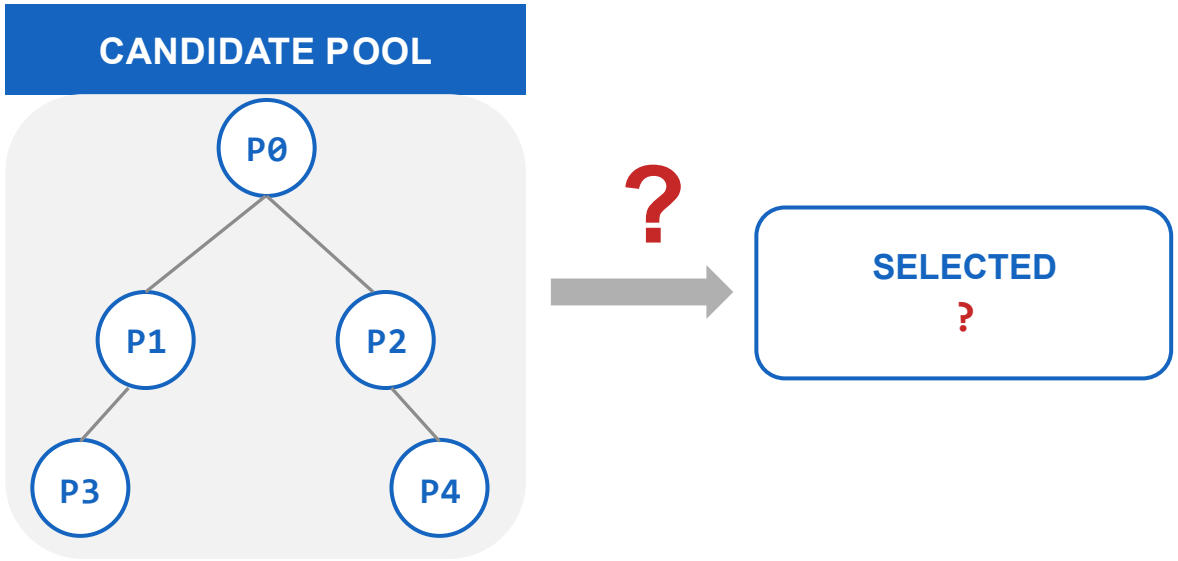
Step 5: evaluate, keep if better

- The child gets a score. It joins the pool **only if it beats its parent**.
- The pool now has two prompts, linked parent to child.

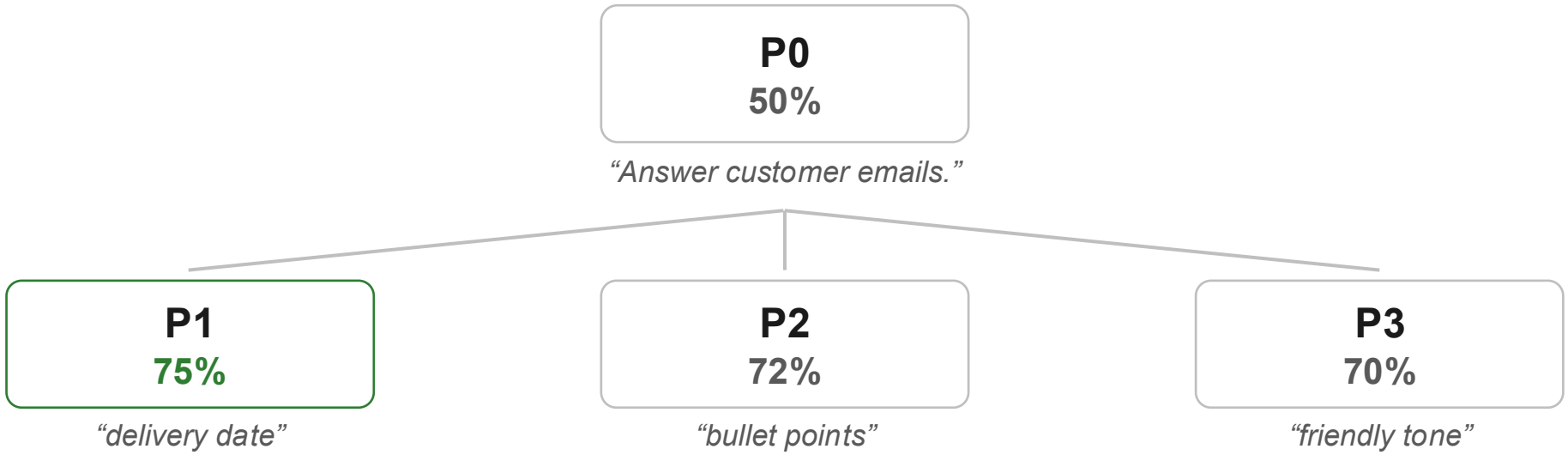


The loop turns: which prompt next?

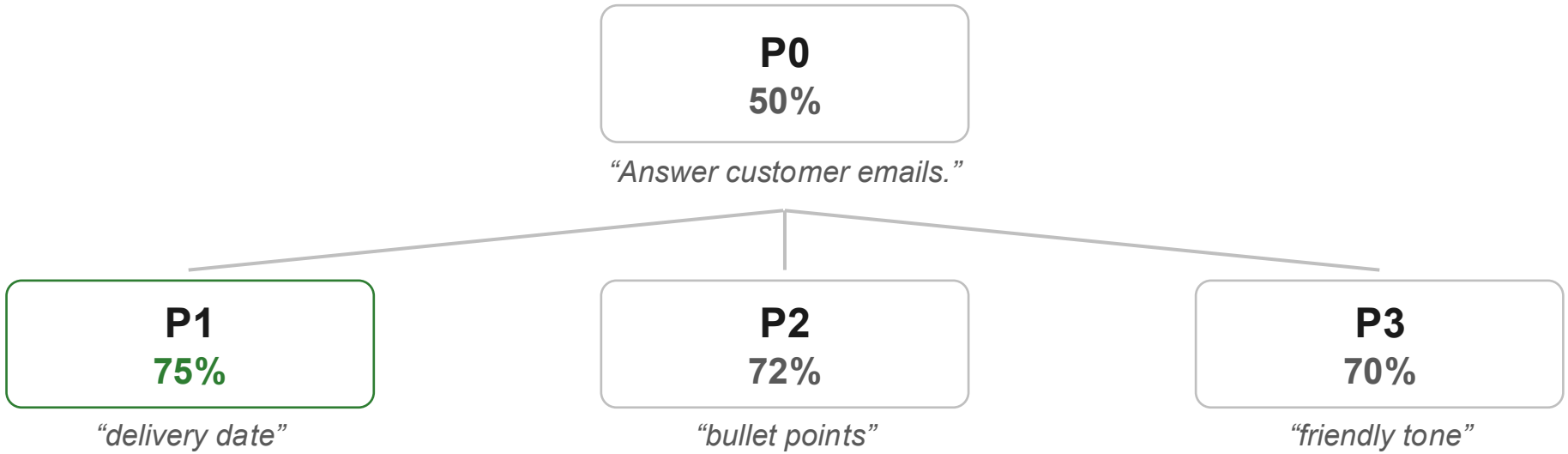
- Repeat the loop and the pool grows into a **family tree** of prompts.
- New problem: with many prompts, **which one should we improve next?**



Choosing the best?



Choosing the best?



P1a 76% **P1b 75%** **P1c 74%** **P1d 76%**

all variants cluster near 76%

Choosing the best?

P0
50%

“Answer customer emails.”

P1
75%

“delivery date”

refine only P1

P1a 76% **P1b 75%** **P1c 74%** **P1d 76%**

all variants cluster near 76%

Choosing only the top scorer commits to one direction, and a better path never gets explored.

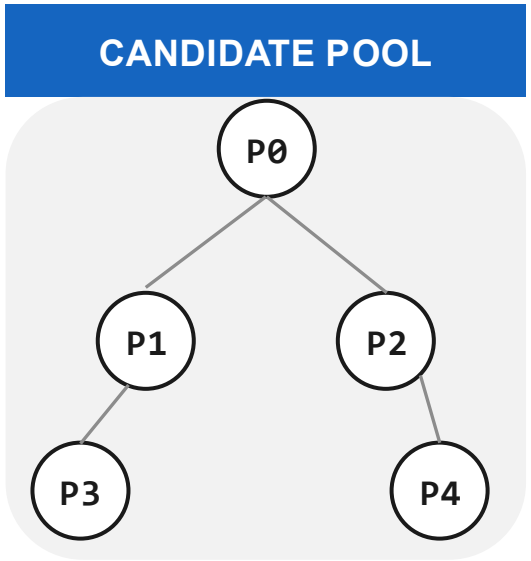
GEPA

PARETO

Find test-case winners first, combine them later.

Pareto-based candidate filtering

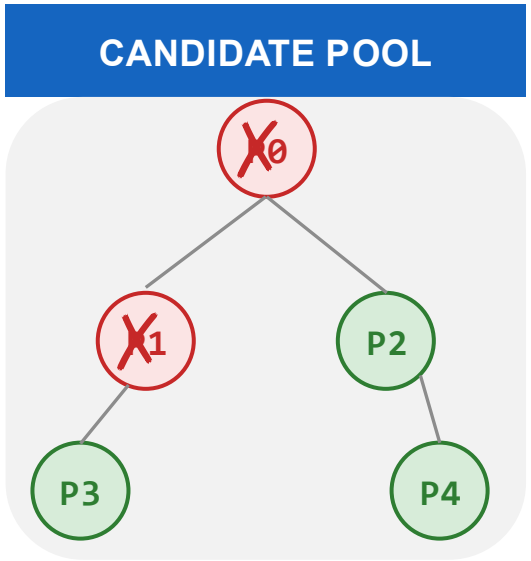
- Keep any prompt that is best on **at least one task** and drop others.



POOL	Task 1	Task 2	Task 3
P0	0.5	0.4	0.6
P1	0.6	0.5	0.5
P2	0.9	0.5	0.6
P3	0.6	0.8	0.7
P4	0.7	0.6	0.9

Pareto-based candidate filtering

- Keep any prompt that is best on **at least one task** and drop others.

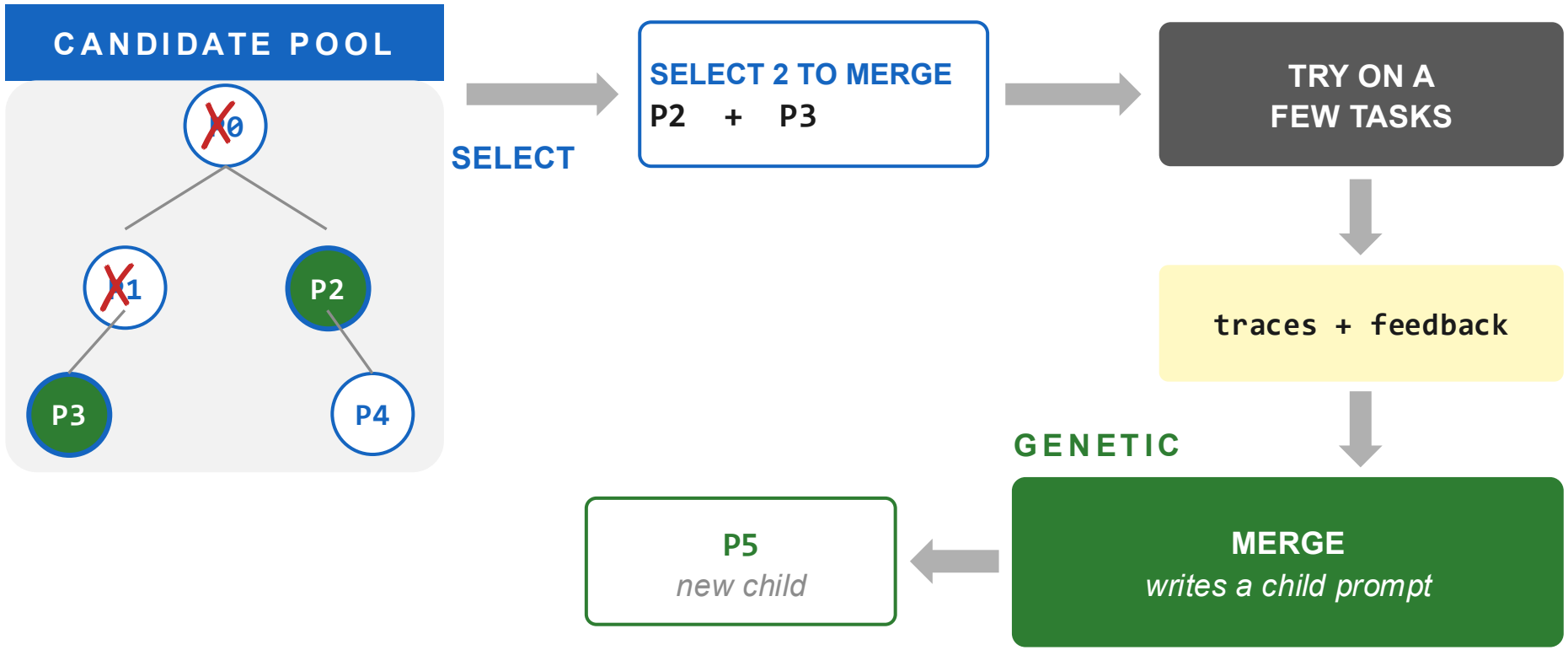


POOL	Task 1	Task 2	Task 3
P0	0.5	0.4	0.6
P1	0.6	0.5	0.5
P2	**0.9 ▲**	0.5	0.6
P3	0.6	**0.8 ▲**	0.7
P4	0.7	0.6	**0.9 ▲**

▲ = best on that task. P0 & P1 win nowhere → eliminated.

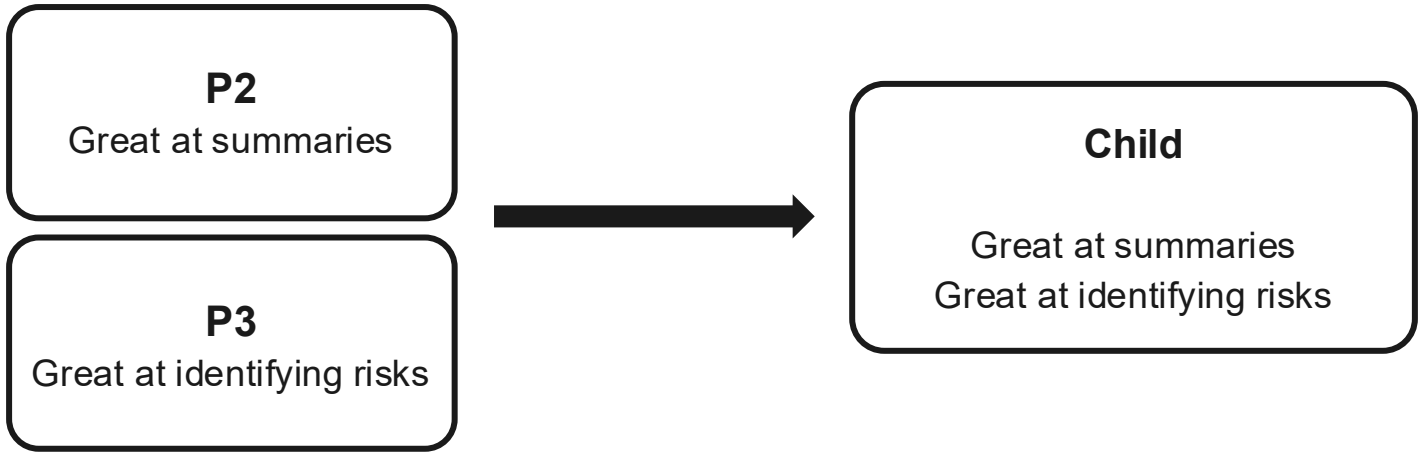
GEPA loop

- This round the system **selects two nodes from the tree and merges them**.

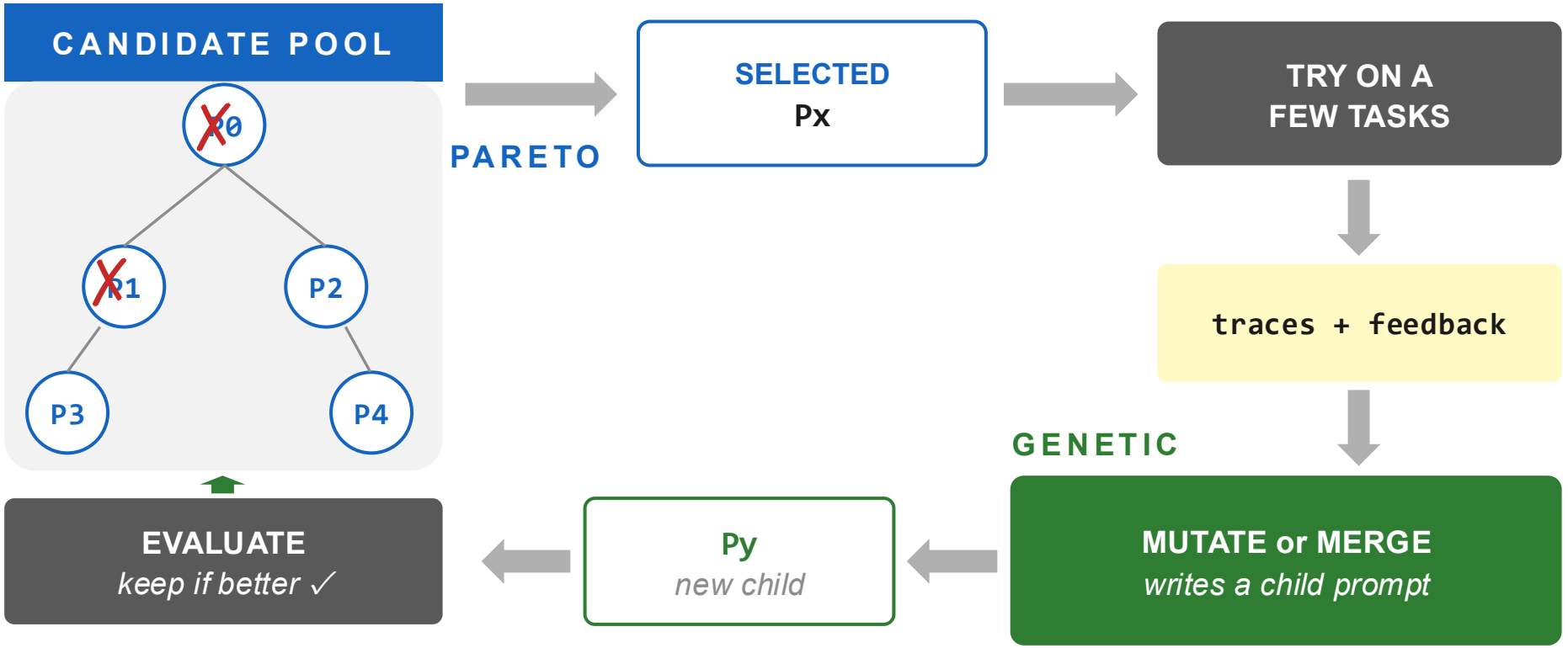


Merge

- Select two successful prompts and combine their strengths into a new prompt.



The full GEPA loop



Outline

1. Why prompts matter
2. Prompt engineering
3. Prompt optimization
4. GEPA’s richer signals: traces and feedback
5. The GEPA loop
6. **Summary and takeaway**

GEPA

Optimize a prompt from language, not just a score.

1 **Collect the trace**
What the AI did: reasoning, tool calls, intermediate outputs.

2 **Read the feedback**
Always a score; often traces; sometimes rubric or human comments.

3 **Mutate the prompt**
Two ways: mutate one prompt from feedback, or merge two winners.

4 **Filter on a Pareto frontier**
Keep a diverse set of winners, not just the single top scorer.

Takeaway: language-driven optimization finds better prompts with far fewer loops.

Thank you